## EXPERIMENT 3

## I2C-BASED HEALTH MONITORING NODE FOR PULSE AND TEMPERATURE ACQUISITION

**CODE:**

```
#include <Wire.h>

#define LM75_ADDRESS 0x48
#define PULSE_PIN 8

unsigned long pulseCount = 0;
unsigned long lastTempTime = 0;
unsigned long lastBpmTime = 0;

int previousPulseState = LOW;

float readTemperature()
{
  Wire.beginTransmission(LM75_ADDRESS);
  Wire.write(0x00);
  Wire.endTransmission();

  Wire.requestFrom(LM75_ADDRESS, 2);

  if (Wire.available() >= 2)
  {
    byte msb = Wire.read();
    byte lsb = Wire.read();

    int16_t rawTemperature =
      ((int16_t)msb << 8) | lsb;

    return rawTemperature / 256.0;
  }

  return -999.0;
}
```

```
void setup()
{
  Serial.begin(9600);
  Wire.begin();

  pinMode(PULSE_PIN, INPUT);

  Serial.println("I2C HEALTH MONITORING NODE");
  Serial.println("--------------------------");
}

void loop()
{
  unsigned long currentTime = millis();

  /* Pulse counting */

  int currentPulseState = digitalRead(PULSE_PIN);

  if (currentPulseState == HIGH &&
      previousPulseState == LOW)
  {
    pulseCount++;
  }

  previousPulseState = currentPulseState;

  /* Temperature acquisition every 1 second */

  if (currentTime - lastTempTime >= 1000)
  {
    lastTempTime = currentTime;

    float temperature = readTemperature();

    Serial.print("Temperature: ");

    if (temperature > -100)
    {
```

```
      Serial.print(temperature, 1);
      Serial.println(" C");
    }
    else
    {
      Serial.println("Sensor Error");
    }
  }

  /* BPM calculation every 5 seconds */

  if (currentTime - lastBpmTime >= 5000)
  {
    lastBpmTime = currentTime;

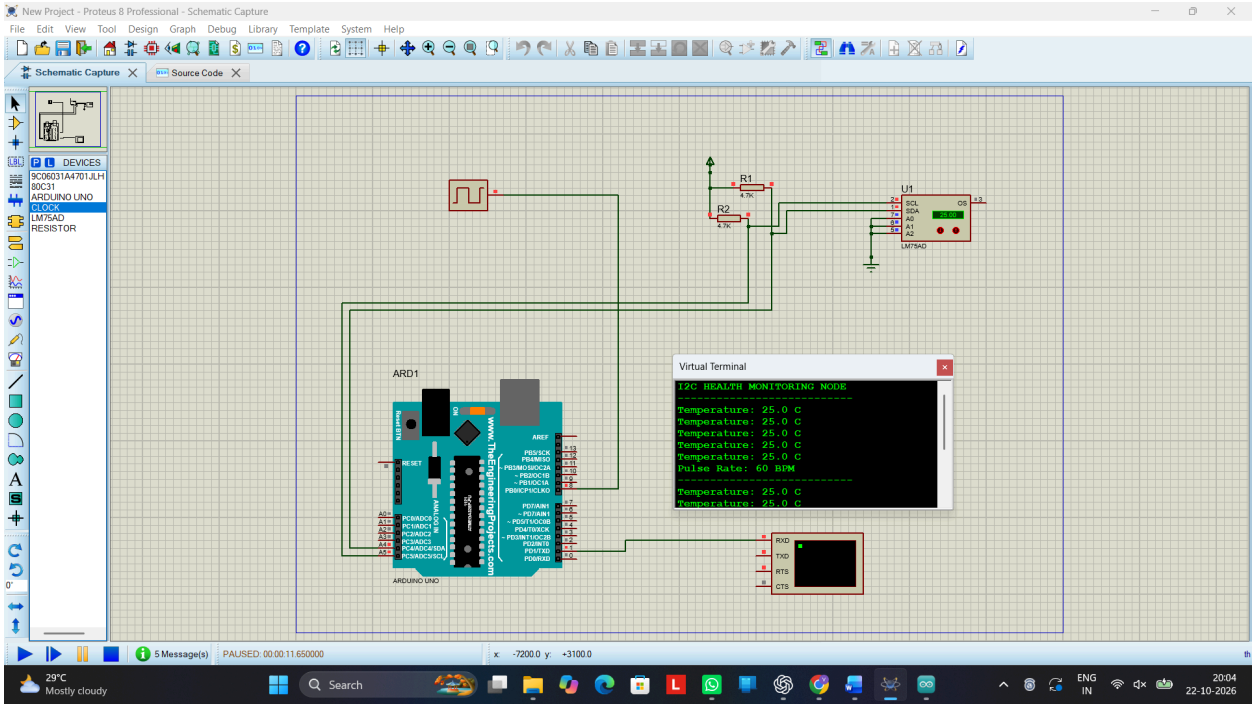
    float bpm = (pulseCount * 60.0) / 5.0;

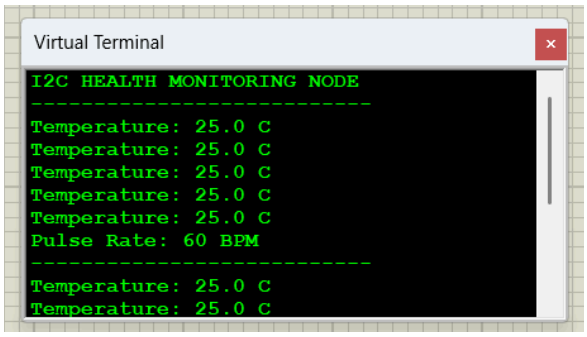
    Serial.print("Pulse Rate: ");
    Serial.print(bpm, 0);
    Serial.println(" BPM");

    Serial.println("----------------------------");

    pulseCount = 0;
  }
}
```

**OUTPUT:**




**REFLECTION:**

I learned about I2C communication and interfacing the LM75AD temperature sensor with Arduino UNO. I understood how to acquire temperature data using the Wire library and measure pulse signals using digital input. I also learned to calculate pulse rate in BPM and display the temperature and pulse rate through the Virtual Terminal. The experiment helped me understand sensor interfacing and serial communication in embedded systems.